Ocean Loop Landing Page

Technical Reference for New Engineers

Last updated: March 2026 • index.html (single file SPA)

1. Purpose & Product Context

Ocean Loop is a subscription service operated by OV Loop Inc. that keeps Fisker Ocean electric vehicles alive and improving after Fisker's bankruptcy. The company provides continued OS updates, mobile app access (Phone-as-Key, remote controls, battery status), and on-staff Fisker engineer support to existing Ocean owners who would otherwise have no official software support path.

The landing page is the primary marketing and conversion surface for this product. Its goals are:

- Explain the value proposition clearly to Fisker Ocean owners who may be worried about their vehicle's future.
- Drive sign-ups at $499/year via a frictionless checkout flow powered by Outseta.
- Surface OS update history and feature details to help owners understand what they get.
- Provide support, FAQ, and contact access so prospects can get answers before committing.
- Handle authenticated user sessions — login and account management — without leaving the site.

2. Architecture Overview

Single-File SPA

The entire site lives in one file: index.html. There is no build system, no bundler, no framework, and no server-side rendering. Everything — HTML structure, CSS styles, JavaScript routing logic, and embedded assets (including the logo as a base64 data URI) — is contained in this single file.

This was a deliberate choice for simplicity and portability. The file can be deployed to any static hosting provider (currently Vercel) by simply uploading or committing it. There are no dependencies to install and no build step to run.

KEY IMPLICATION FOR ENGINEERS

When editing the site, you are editing index.html directly. Use the section comment headers (e.g. ═══ HOME ═══, ═══ CHECKOUT ═══) to navigate to the right page. Each page is a self-contained div with its own nav, content, and footer — do not accidentally edit the wrong page's nav copy.

Client-Side Routing

Navigation between pages does not load a new document. Instead, JavaScript shows and hides divs with the class "page". The router works as follows:

- On load, the entry point reads window.location.pathname, maps it to a page ID via the ROUTES object, calls history.replaceState, then calls navigate().
- navigate(id) hides all .page elements, adds the "active" class to the target page, updates the active nav link highlight, calls history.pushState to update the URL, and scrolls to the top.
- showPage() is an alias for navigate() kept for backward compatibility with existing onclick= attributes.
- The popstate event listener handles browser back/forward navigation by calling navigate() without pushing a new history entry.

ROUTES	Maps URL paths (/login, /pricing, etc.) → page IDs
SLUGS	Maps page IDs → URL paths (the reverse of ROUTES)
navigate(id)	Core routing function — always call this to change pages
showPage(id)	Alias for navigate() — legacy, kept for onclick= compatibility

Note: pushState throws a SecurityError in file:// contexts or sandboxed iframes. The navigate() function wraps this in a try/catch so routing still works visually even if the URL cannot be updated.

3. Page Inventory

There are 10 pages. Each lives in a div with the ID pattern page-{id} and the class "page". Only the active page has the additional class "active" (which sets display: block; all others are display: none).

Page ID	URL	Purpose
`home`	`/`	Hero, feature highlights, OS update previews, pricing teaser, FAQ snippet, CTA
`features`	`/features`	Full feature breakdown: app, OS updates, support, remote diagnostics
`updates`	`/updates`	Complete OS release history with pricing, changelogs, and upgrade paths
`pricing`	`/pricing`	Subscription plan details, one-time update pricing, and comparison table
`support`	`/support`	FAQ accordion, support contact options, engineer team info

Page ID	URL	Purpose
contact	/contact	Contact form and direct support channel links
checkout	/checkout	Outseta registration embed — sign-up / payment flow
login	/login	Outseta login embed — returning user sign-in
profile	/profile	Outseta profile embed — subscription, billing, account management
home	/home	Alias for / — routes to the same home page

Each page div contains its own fully duplicated nav bar and footer. This is intentional — it ensures that the nav's "active" link highlight is correct per page without requiring JavaScript to update it on every navigate. If you edit nav link labels or add a new nav item, you must update it in every page's nav (10 navs total), plus the mobile nav drawer inside each.

4. Navigation

Desktop Nav

Each page contains a sticky nav bar with the Ocean Loop logo, a horizontal link list, and two CTA buttons on the right: Login and Sign Up. Nav links call showPage() or navigate() via onclick= attributes. The active link is pre-set with class="active" in each page's own nav markup rather than being set dynamically by JS.

Mobile Nav

A hamburger button (class nav-hamburger) toggles a slide-down drawer (class mobile-nav-drawer) via the toggleMobileNav() function. The drawer contains the same links as desktop plus Login/Sign Up buttons. Each link calls closeMobileNav() after navigating. The mobile nav state (open/closed) is tied to the hamburger button's "open" CSS class.

Auth-Aware Buttons

The updateNavAuth() function checks whether Outseta has a logged-in user and shows/hides the Login and Sign Up buttons accordingly. It also shows a btn-account button (currently present in the markup but hidden by default) for logged-in users. This function is called after every navigate() and on a 300ms timeout after initial page load to give Outseta time to initialize.

5. Outseta Integration

Outseta is the all-in-one platform handling authentication, subscriptions, billing, and CRM. It is loaded via a CDN script tag and configured with the o_options object.

OUTSETA CONFIGURATION

Domain	ov-loop.outseta.com
Load list	auth, customForm, emailList, leadCapture, profile, support
nocode	NOT loaded — this is critical. The nocode module hijacks SPA routing and must stay off.
callback	Empty function — do not use this to trigger navigation. Only use it for auth state updates.

Embed Widgets

Three Outseta embed widgets are used, each on a dedicated page:

Checkout (register) — /checkout

```
data-o-auth="1"  data-widget-mode="register"
data-plan-uid="E9LXwZ9w"  data-plan-payment-term="year"
data-skip-plan-options="true"  data-mode="embed"  data-o-spa="0"
```

Drops users directly into the annual plan registration flow. The plan UID hardcodes the $499/year subscription.

Login — /login

```
data-o-auth="1"  data-widget-mode="login"
data-authentication-callback-url="/profile"  data-mode="embed"  data-o-spa="0"
```

Standard login form. On successful auth, Outseta redirects to /profile via the callback URL.

Profile — /profile

```
data-o-profile="1"  data-mode="embed"  data-o-spa="0"
```

Outseta's account management widget — subscription status, billing info, password change, etc.

CRITICAL: DATA-O-SPA="0"

All three Outseta embeds include data-o-spa="0". This tells Outseta not to treat the page as a single-page app and not to intercept routing. Without this attribute, Outseta's internal navigation handling conflicts with our custom router and breaks page transitions. Do not remove it.

Auth State

Outseta widgets render automatically based on their data attributes — no JavaScript initialization calls are needed. The updateNavAuth() function reads the logged-in state by calling Outseta.getUser() and checking for a non-empty Email field. It uses this to toggle visibility of Login/Sign Up vs. Account buttons.

If Outseta is not yet loaded when updateNavAuth() runs (e.g. on first paint), the function returns early safely. The 300ms setTimeout in the entry point is a deliberate delay to allow Outseta to initialize before the first auth check.

6. Styling

Design System

All styles are in a single <style> block in the <head>. CSS custom properties (variables) define the design tokens:

--navy	#0D1B2A — primary dark color, headings, nav background
--blue	#2563EB — links, badges, interactive elements
--orange	#E8572A — accent, CTA buttons, "Latest" badges
--bg	#F0F4F8 — page background
--text-mid	#4B5563 — body text
--white	#FFFFFF

Fonts

Two Google Fonts are loaded: DM Serif Display (used for large display headings/hero text) and Inter (used for everything else — body, nav, labels, buttons). The font-family stack falls back to system-ui if the CDN is unavailable.

Page Show/Hide

The core show/hide mechanism is:

```
.page { display: none; }
.page.active { display: block; }
```

This is the only mechanism controlling page visibility. There are no opacity, visibility, or z-index tricks. If a page is blank, the first thing to check is whether its div is properly a direct child of <body> and not accidentally nested inside another page div (this was a real bug — see the Known Pitfalls section).

Responsive Design

The site is responsive via media queries. Key breakpoints are around 768px for nav (hamburger appears) and various content grids collapsing to single columns. The checkout page has explicit media query overrides to collapse its three-column grid on small screens.

7. Known Pitfalls & History

READ THIS BEFORE DEBUGGING A BLANK PAGE

The Missing Bug (Resolved)

During development, all routes except / rendered as a completely blank page. After extensive debugging (CSS audit, JS routing trace, DOM parent check), the root cause was a missing closing tag at the end of the #page-checkout section. The browser auto-corrected this by nesting all subsequent page divs inside #page-checkout, so when checkout was display:none, all other pages inherited that and collapsed to zero size.

Fix applied: a <!-- /page-checkout --> comment tag was added at line ~1994. If you ever see a blank page again, the first thing to do is check the DOM parent chain in DevTools: every #page-{id} div must be a direct child of <body>.

Outseta nocode Module

The nocode module was originally in the Outseta load list. It was removed because it takes over SPA routing — intercepting clicks and anchor tags in ways that conflict with the custom router. Never add nocode back to the load list.

Outseta Callback

The Outseta callback function (called after Outseta initializes) was originally used to trigger page navigation. This caused double-navigation bugs. The callback is now an empty function. Any auth-related UI updates should go in updateNavAuth(), not the callback.

history.replaceState Order

The entry point calls history.replaceState before calling navigate(). This order matters. If navigate() ran first and pushed a state, then replaceState would overwrite it with stale data. Always maintain this order if modifying the entry point.

Nav Duplication

As described in Section 3, the nav is duplicated inside every page div. This is verbose but intentional. If you are asked to "update the nav," make sure to update all 10 instances. A find-and-replace in a code editor is the safest approach, being careful to only change nav content and not accidentally touch page content below each nav.

---

8. Deployment

The site is hosted on Vercel as a static site. Deployment is a single file drop — no build, no CI pipeline, no environment variables. The file to deploy is index.html.

Vercel injects a small analytics script (213.4f1e2f6...min.js) into the page at runtime. This script has been audited and does not affect page visibility or routing. It is not a source of CSS conflicts.

Because all routes (/login, /pricing, etc.) are handled client-side by the SPA router, Vercel must be configured to serve index.html for all paths. This is typically done with a vercel.json rewrite rule:

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Without this rewrite, direct navigation to a non-root URL (e.g. typing /pricing into the browser) will return a 404 from Vercel instead of loading the SPA.

---

9. Quick Reference

Adding a New Page

- Add a new entry to ROUTES: { "/new-page": "newpage" }
- Add a new entry to SLUGS: { "newpage": "/new-page" }
- Add a new div: ... before the closing </body> tag
- Make sure the div is a direct child of <body> — verify with DevTools after adding
- Add a nav link to all 10 existing page navs (desktop + mobile)

Editing Page Content

- Find the correct page using the ═══ PAGENAME ═══ comment headers
- Content sits between the closing </nav> tag and the opening <footer> tag
- Styles are in the single <style> block at the top — use existing CSS variables

Debugging a Blank Page

- Open DevTools > Elements, find the #page-{id} div, check its parent — it must be BODY
- Check that .page.active is present on the correct div
- Check the console for JS errors that might have halted the navigate() call
- Check that the page div was not accidentally placed inside another page div (unclosed tag)

Working with Outseta

- Do not add nocode to the load list
- Do not use the Outseta callback for navigation — use updateNavAuth() for auth state only
- Always include data-o-spa="0" on any Outseta embed div
- The authentication callback URL on the login embed (/profile) controls where Outseta redirects after login